



西南交通大学
Southwest Jiaotong University

地球科学与环境工程学院

《遥感原理与应用》

课间实验报告书

设计题目： 图像匹配与几何纠正

学 号： 2023113911

班 级： 遥感 1 班

姓 名： 于沛豪

日 期： 2025. 4. 26

目 录

一、实验一---图像匹配	1
1.1 NCC 匹配算法	1
1.2 设计内容	1
1.3 Matlab 算法实现	2
二、实验二---图像匹配与几何纠正	3
2.1 实验流程	3
2.2 格网建立与匹配	3
2.3 一次多项式图像纠正与投影到图像坐标系	4
2.4 双线性采样插值	5
2.5 实验结果	6

一、实验一---图像匹配

1.1 NCC 匹配算法

NCC (normalization cross-correlation) 归一化相关性匹配算法，在图像匹配领域占据重要地位，对噪声具有一定的鲁棒性，不受亮度变化的影响，但是它对存在旋转的图像不能进行匹配。

NCC 算法是区域匹配算法，就可以对特征区域部分进行匹配，可以提高单个特征匹配的精度，还能提升整体模型的速度，同时抵消灰度变化带来的影响，算法如下所示：

$$NCC = \frac{\sum_i^N \sum_j^N (a_{ij} - \bar{a})(b_{ij} - \bar{b})}{\sqrt{\sum_i^N \sum_j^N (a_{ij} - \bar{a})^2 \sum_i^N \sum_j^N (b_{ij} - \bar{b})^2}}$$

NCC 算法首先规定一个一定大小的模板图像灰度矩阵，让它在参考图像的所有可能的灰度阵列中，按某种相似性度量方法进行逐像素的搜索比较，从而进行匹配。越接近 1 说明相关性越大，说明两幅图像越相似，反之反之相似度越低，导致无法匹配。因此 NCC 输出结果的好坏，即可设置一个阈值，通过比较来确定。

1.2 设计内容

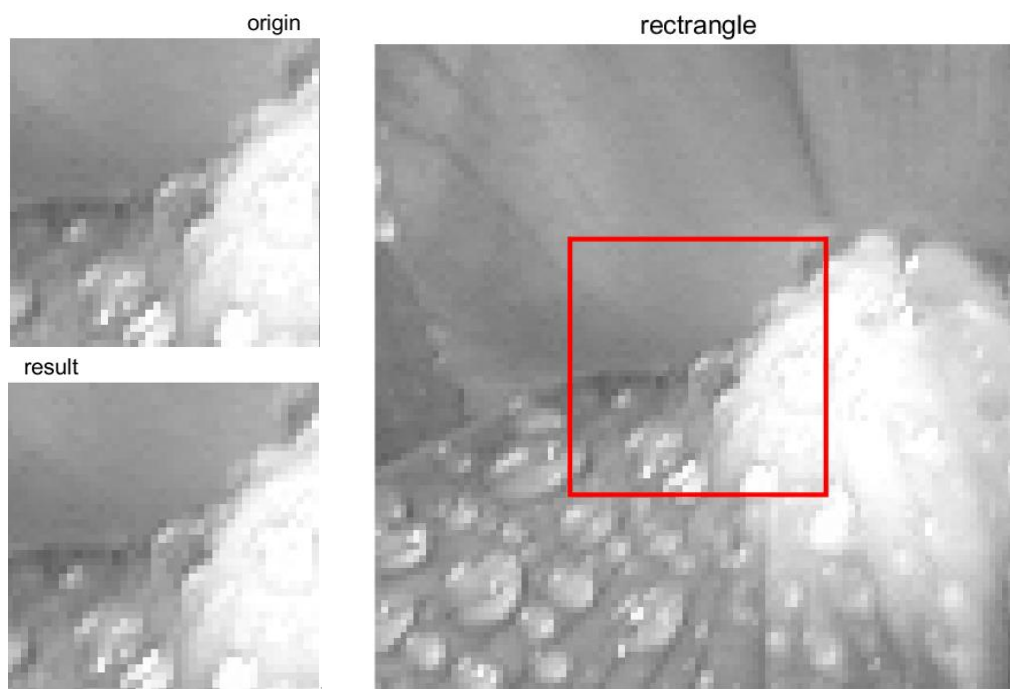


图 1 实验一结果图像

1.3 Matlab 算法实现

给定一个小图（input）和一张大图（reference），在大图中滑动小图，在每一个位置计算归一化互相关系数（NCC 值），输出一个矩阵 NCC，矩阵的每一个元素表示对应位置的匹配程度，越接近 1 表示越相似。

其中 $\text{size}(\text{reference}, 1)$ 是参考图像的高度， $\text{size}(\text{input}, 1)$ 是模板图像的高度，所以整体大小是（参考高度 - 模板高度） $\times$ （参考宽度 - 模板宽度），使得模板在参考图像内滑动。

```
function NCC = match_ncc(input,reference)
    a_equal = mean(input,"all");
    NCC = zeros(size(reference,1)-size(input,1),size(reference,2)-
size(input,2));
    for addi = 0:size(reference,1)-size(input,1)
        for addj = 0:size(reference,2)-size(input,2)
            b_equal = mean(reference(1+addi:addi+size(input,1),1+addj:addj+size(input,2)),"all");
            up_array = (reference(1+addi:size(input,1)+addi,1+addj:size(input,2)+addj) - b_equal).*(input(1:size(input,1),1:size(input,2)) - a_equal);
            up = sum(up_array,"all");
            down1_array = (input(1:size(input,1),1:size(input,2)) - a_equal).^2;
            down1 = sum(down1_array,"all");
            down2_array = (reference(1+addi:size(input,1)+addi,1+addj:size(input,2)+addj) - b_equal).^2;
            down2 = sum(down2_array,"all");
            down = sqrt(double(down1*down2));
            NCC(addi+1,addj+1) = up/down;
        end
    end
end
```

在代码原实现时，没有考虑算法复杂度，在完成图像矩阵求和以及其他操作时，使用了逐个计算的形式，这使得时间复杂度达到了 n^4 ，从而在实际计算时跑了很长时间都跑不出代码。

经过研究后，对 NCC 算法的求和等操作采用了矩阵操作，也算发挥了 Matlab 的特长。使得程序可以极为迅速的运行出最终的结果。

此处 Input 为小图，无需进行格网匹配即可得到匹配结果，又因算法实现迅速，可在 for 循环的步长中设置为 1，无需指定更长步长。

二、实验二---图像匹配与几何纠正

2.1 实验流程

实验二相比于实验一的匹配更难一步，即需要建立格网，将大的图片区分为小的模板进行匹配。

此处将实验二分成了三个部分，分别是格网建立与 NCC 匹配，图像变换，线性插值，流程图如下所示：

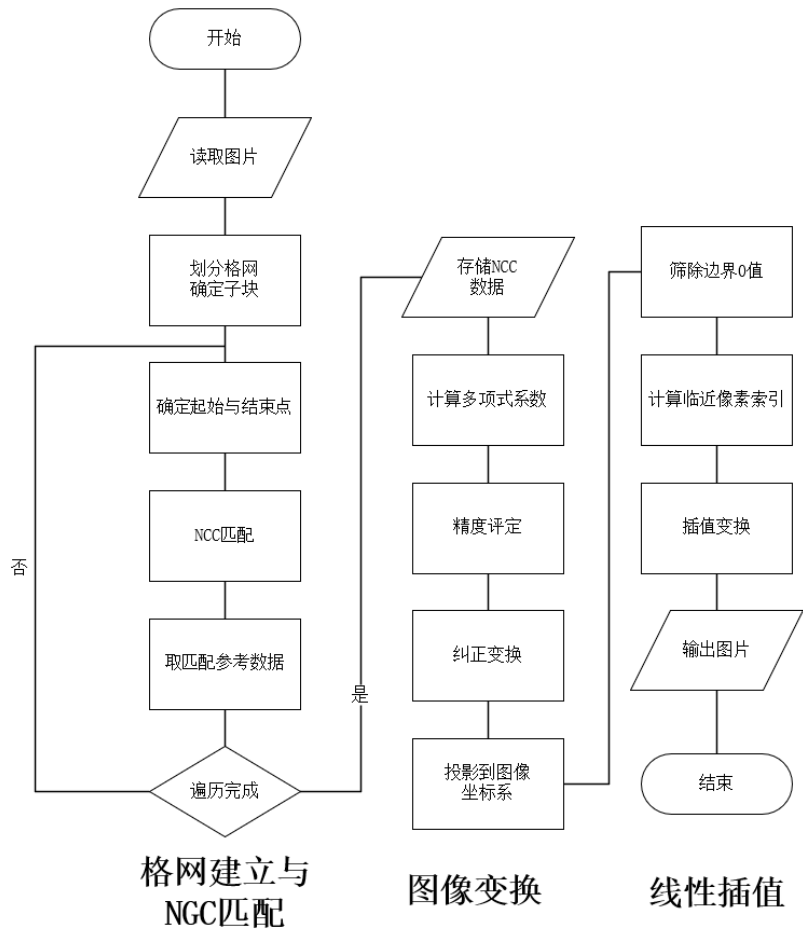


图 2 图像匹配与几何纠正流程图

2.2 格网建立与匹配

NGC 匹配算法同实验一，此处不再赘述。

格网建立时首先取起始点与结束点，寻找其索引最大/小值，确保索引不越界，故使用了 max/min 函数。

调用 match_ncc 对划分的格网小块作为 input 进行格网匹配，取匹配参考图块作为绘图展示，之后写入匹配值 ncc_result，包括匹配前 xy 坐标与匹配后 xy 坐标

格网建立与匹配函数如下所示：

```

for xloc = 2:9
    for yloc = 2:9
        % 起始点
        x_minloc = round(x(xloc));
        y_minloc = round(y(yloc));

        % 结束点，确保不越界
        x_maxloc = min(x_minloc + sub_h - 1, size(img,1));
        y_maxloc = min(y_minloc + sub_w - 1, size(img,2));
        batch = img(x_minloc:x_maxloc, y_minloc:y_maxloc);

        % 匹配
        ncc = match_ncc(batch, reference);
        [M, idx] = max(ncc(:));
        [row, col] = ind2sub(size(ncc), idx);

        ncc_result = [ncc_result, [x_minloc ; y_minloc ; row ; col]];

        % 取匹配参考图块
        ref_x_end = min(row + sub_h - 1, size(reference,1));
        ref_y_end = min(col + sub_w - 1, size(reference,2));

        matched_patch = reference(row:ref_x_end, col:ref_y_end);
        all_image = [all_image; [matched_patch batch]];

        disp(['已完成第', num2str(index), '个点的匹配，该点 NCC: ', num2str(M)]);
        index = index + 1;
    end
end

```

2.3 一次多项式图像纠正与投影到图像坐标系

读取两幅影像和特征点数据，利用一阶多项式建立参考影像到待配准影像的坐标映射关系，并计算变换精度。然后根据拟合出的参数，将待配准影像变换到新坐标系，生成配准结果。

图像纠正与投影到图像算法代码如下所示：

```

%% 多项式的遥感图像纠正
Lx = x';
Ly = y';
% 计算多项式系数
delta_a = (A'*A)\A'*Lx';
delta_b = (A'*A)\A'*Ly';

Vx = A*delta_a-Lx';
Vy = A*delta_b-Ly';
% 精度评定
epsilon_x = sqrt(sum(Vx'.*Vx,"all")/61);
epsilon_y = sqrt(sum(Vy'.*Vy,"all")/61);
disp(['X 精度为',num2str(epsilon_x),', Y 精度为',num2str(epsilon_y)])
%% 遥感图像纠正变换
conner_x = delta_a(1)+delta_a(2)*[1;m;m;1]+delta_a(3)*[1;1;n;n];
conner_y = delta_b(1)+delta_b(2)*[1;m;m;1]+delta_b(3)*[1;1;n;n];
X1 = min(conner_x,[],'all');
X2 = max(conner_x,[],'all');
Y1 = min(conner_y,[],'all');
Y2 = max(conner_y,[],'all');
M = (Y2-Y1)+1;
N = (X2-X1)+1;

for i = 1:o
    for j = 1:p
        convert_x = delta_a(1)+delta_a(2).*i+delta_a(3).*j;
        convert_y = delta_b(1)+delta_b(2).*i+delta_b(3).*j;
        %原图像灰边
        convert(floor(convert_x+45),floor(convert_y+45)) = I(i,j);
    end
end
end

```

2.4 双线性采样插值

对变换后影像 `convert_img` 中灰度值为 0 的像素进行插值修补。首先找出图像中值为 0 的像素点，排除图像边界上的点，只保留内部的空洞点。然后，对每个空洞像素，取它上下左右四个邻居的灰度值，计算平均值填补原本为 0 的位置，最后显示插值后的图像。

核心代码如下所示

```

[a,b] = size(convert_img);
[zero_y, zero_x] = find(convert_img == 0);

% 筛除边界的 0 值
valid_idx = (zero_x > 1) & (zero_x < b) & (zero_y > 1) & (zero_y < a);
afterzero_x = zero_x(valid_idx);
afterzero_y = zero_y(valid_idx);

% 邻近像素索引
left_x = afterzero_x - 1;
right_x = afterzero_x + 1;
up_y = afterzero_y - 1;
down_y = afterzero_y + 1;

% 插值计算
convert_img(sub2ind([a, b], afterzero_y, afterzero_x)) = ...
    0.25 * convert_img(sub2ind([a, b], up_y, afterzero_x)) + ...
    0.25 * convert_img(sub2ind([a, b], down_y, afterzero_x)) + ...
    0.25 * convert_img(sub2ind([a, b], afterzero_y, left_x)) + ...
    0.25 * convert_img(sub2ind([a, b], afterzero_y, right_x));

figure(3);
imshow(uint8(convert_img));
title('chazhi');

```

2.5 实验结果

在变换后，未经插值的图像如下所示，其中右图为左图放大

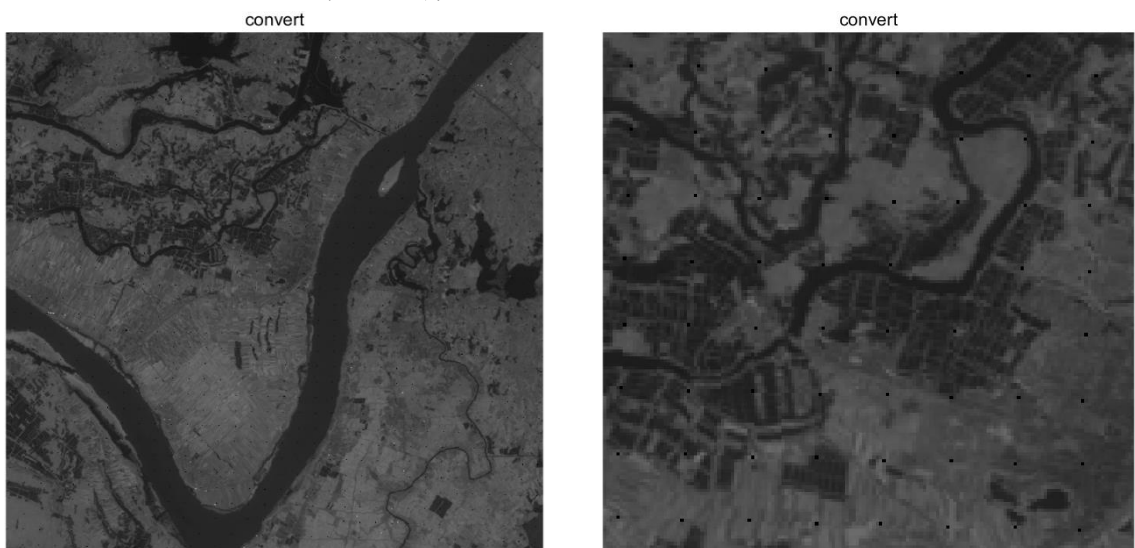


图2 一次多项式图像纠正与其局部放大图

这是因为一次多项式插值后，会有部分投影点因四舍五入而空缺，因此显示出黑色，故需要双线性采样插值，插值后如下所示

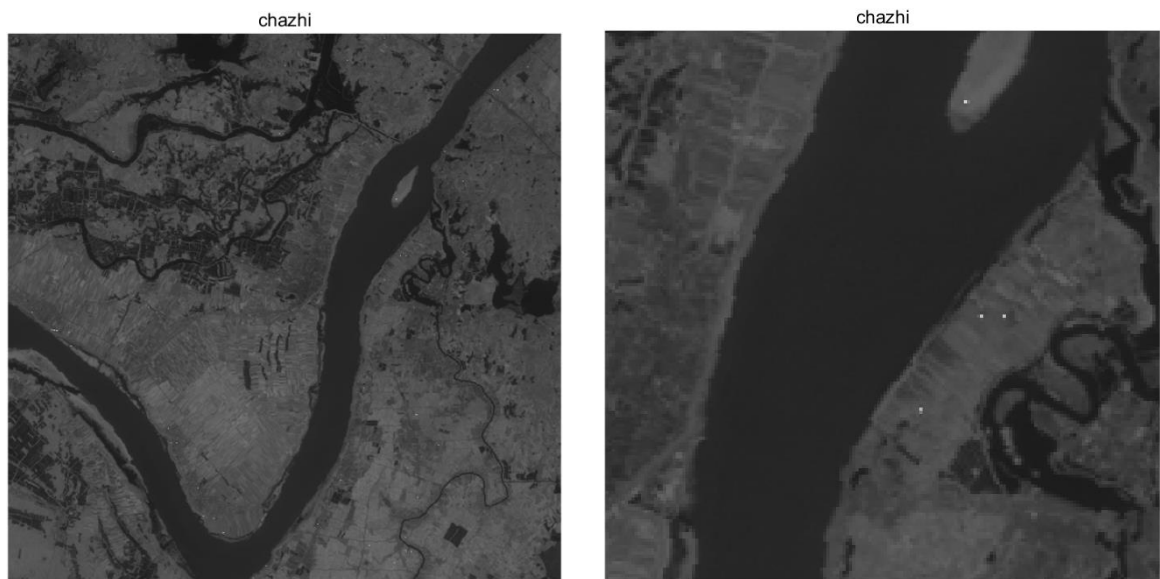


图3 双线性采样插值与其局部放大图

可见，实验结果表明较好的完成了实验任务。

实验完整代码见文件，其中 Match3 为实验 3 代码，Match4_1 与 Match4_2 为实验四代码，Match4_1 为格网匹配计算并输出同名点坐标，Match4_2 为一次多项式变换与双线性插值。